

Reporte Técnico: Comportamiento del Sistema Operativo

Análisis reflexivo y justificación teórica de la práctica de comandos

1. Resumen de los Comandos Ejecutados

Durante la práctica se interactuó directamente con el sistema de archivos de Linux mediante la terminal, siguiendo este flujo de trabajo:

```
# Creación de carpetas iniciales
mkdir practica1 practica2

# Creación del archivo de texto en practica1
cd practica1
gnome-text-editor Readme.txt

# Verificación de metadatos
ls -l

# Duplicación del archivo hacia practica2
cp ~/Documents/practica1/Readme.txt ~/Documents/practica2/

# Subcarpetas y archivo interno en practica2
cd ~/Documents/practica2
mkdir vacia info
cd info
gnome-text-editor Readme.txt

# Copia recursiva de carpetas hacia practica1
cp -r ~/Documents/practica2/vacia ~/Documents/practica1
cp -r ~/Documents/practica2/info ~/Documents/practica1

# Eliminación de archivo y carpeta en practica1
cd ~/Documents/practica1
rm Readme.txt
rm -r info
```

2. Justificación Teórica de las Operaciones

- **Creación de carpetas (`mkdir`):** En Linux, los directorios son en realidad archivos especiales que guardan una lista de nombres asociados a números de *inodo* (la estructura donde el sistema guarda los metadatos). Al crear una carpeta, el sistema operativo le asigna un inodo y crea de forma automática dos entradas: `.` (referencia a sí misma) y `..` (referencia a la carpeta que la contiene).

- **Lectura de metadatos (`ls -l`):** Este comando no abre el contenido del archivo, sino que consulta directamente los atributos del inodo: permisos de acceso (lectura, escritura y ejecución para dueño, grupo y otros), usuario propietario, tamaño en bytes y fecha de última modificación.
- **Copia de archivos y carpetas (`cp` y `cp -r`):** Al usar `cp` se duplica la información creando un inodo totalmente nuevo e independiente. La opción `-r` (recursiva) es obligatoria al copiar carpetas completas porque el sistema necesita recorrer todo el árbol interno de archivos y subcarpetas para reconstruir la estructura en el destino.
- **Eliminación directa (`rm` y `rm -r`):** A diferencia del entorno visual donde los archivos van a la papelera de reciclaje, el comando `rm` desvincula el archivo de inmediato mediante la llamada `unlink()`. Si ningún programa lo está usando, el sistema operativo marca ese espacio como libre para sobrescribirse, por lo que el borrado es definitivo y sin confirmaciones intermedias.

3. Análisis Reflexivo: ¿Por qué falló `sudo gedit` ?

Al intentar ejecutar `sudo gedit Readme.txt` el sistema generó un error o bloqueo. Este comportamiento tiene tres causas técnicas clave:

- **Uso de Wayland en lugar de X11:** Las versiones actuales de Ubuntu utilizan el servidor gráfico Wayland por razones de seguridad. Wayland no permite por diseño que aplicaciones con permisos de administrador (`sudo` / root) se comuniquen directamente con la pantalla gráfica del usuario común, bloqueando la apertura del programa.
- **Riesgo de daño en permisos del usuario:** Abrir programas visuales con `sudo` dentro de tu carpeta personal hace que los archivos de configuración interna se guarden como propiedad del usuario ``root``. Esto suele provocar que más adelante el usuario regular no pueda abrir programas o tenga problemas para iniciar sesión.
- **Principio de menor privilegio:** Para crear o modificar un archivo dentro de tus propios documentos personales (`/home/usuario/Documents`) no se requieren permisos de administrador. La solución correcta es abrir el editor normal sin `sudo` usando el editor predeterminado del sistema (`gnome-text-editor`) o un editor de terminal ligero (`nano`).

4. Conclusión Breve

La práctica demuestra que trabajar en la terminal no es solo escribir texto, sino interactuar de forma directa con los permisos y el almacenamiento del núcleo del sistema operativo. Comprender el porqué de cada comando permite resolver problemas habituales, evitar dañar permisos del sistema con `sudo` innecesario y operar con mayor rapidez y seguridad.